

國立清華大學
National Tsing Hua University

碩士論文
Master's Thesis

這裡填入論文中文題名
Type English Title Here

系所別：資訊安全研究所

Dept. / Grad. Inst.: Institute of Information Security

學號 Student ID: 111164529

研究生 Author：黃宏勝 Hong-Sheng Huang

指導教授 Advisor：

中華民國一一五年七月

July 2026

國立清華大學
National Tsing Hua University

碩士論文
Master's Thesis

這裡填入論文中文題名
Type English Title Here

系所別：資訊安全研究所

Dept. / Grad. Inst.: Institute of Information Security

學號 Student ID: 111164529

研究生 Author：黃宏勝 Hong-Sheng Huang

指導教授 Advisor：

中華民國一一五年七月

July 2026

摘要

摘要是整篇論文的濃縮總結，需精簡扼要，涵蓋論文核心內容，邏輯連貫地總結各章節要點，吸引讀者興趣。建議控制在一頁內，約400–600字，避免使用圖表。

內容要素：

- 研究動機與目標：闡述研究背景、核心問題及主要目標。
- 研究範圍：明確界定研究的課題與範疇。
- 研究方法：概述方法、技術或工具，並簡述其合理性。
- 主要結果：突出核心發現或數據。
- 結果比較與意義：將結果與研究目標及文獻比較，凸顯貢獻。
- 未來影響與建議：說明研究對學術或實務的助益，提出未來方向。

撰寫建議：

- 使用精煉語言，確保非領域專家也能理解。
- 完成論文其他部分後再提煉摘要，確保內容全面。
- 使用純文字格式（無HTML或特殊格式），便於圖書館建檔與網路檢索。
- 附上5–7個關鍵詞，提升論文檢索率。

關鍵字：論文格式、圖書館建檔、範例、提煉、圖書館建檔

Abstract

The abstract is a concise summary of the dissertation, encapsulating its core content in a clear, logically coherent manner to engage readers. It should be limited to one page, approximately 300–500 words, and avoid including figures or tables.

Content Elements

- **Research Motivation and Objectives:** Describe the background, core research question, and primary goals.
- **Research Scope:** Clearly define the topics and boundaries of the study.
- **Methodology:** Summarize the methods, techniques, or tools used and briefly justify their selection.
- **Key Results:** Highlight the most significant findings or data.
- **Comparison and Significance:** Compare results with the initial objectives and existing literature, emphasizing contributions.
- **Impact and Future Directions:** Discuss the study's benefits to academic or practical fields and suggest future research or applications.

Writing Tips

- Use precise, accessible language to ensure clarity for non-experts.
- Draft the abstract after completing other sections to ensure it fully reflects the dissertation's content.
- Format in plain text (no HTML or special formatting) to comply with library archiving and online search requirements.

Keywords: database, HTML, library archiving, dissertation, methodology

致謝

首先，我想感謝周彤老師在這個 project 裡給予我許多幫助，不論從程式架構的建議，或是研究方向上的引導，無疑都為整個研究帶給我可靠的支持。可以說，沒有他的協助，今天就不會有這篇論文的誕生。

接著，在這幾年的碩士生涯中，我的指導教授孫宏民老師提供給我一個很自由的實驗室環境，使我可以充分探索自己有興趣的題目。除此之外，孫老師也總是會確保實驗室擁有充足的研究資源，讓我在進行研究時毫無後顧之憂，很感謝他的幫忙。

最後我想感謝我的母親長期以來給我的關心與支持，沒有她的照顧，今日我可能無法走到這一步，希望這一份小小的成就，可以讓她引以為傲；本論文獻給我無與倫比的母親，她一直堅信我能突破困境，發出耀眼光芒。

Contents

摘要	i
Abstract	i
致謝	ii
Table of Contents	iv
List of Figures	v
List of Tables	vi
Nomenclature	1
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Contributions	2
1.4 Organization	2
2 Preliminaries	3
2.1 Notations	3
2.2 Quantum Circuits	3
2.3 Optimization Target	4
2.4 Grover's Search Algorithm	4
3 Existing Approach for CNOT Gate Composition	5
3.1 De Brugière et al. Greedy Method	5
3.2 Shi & Feng Greedy Method	5
3.3 Matrix Cost Function	5
3.3.1 Log	7

3.3.2	Sum	7
3.3.3	Square	7
4	Our Greedy Algorithm	8
4.1	Row-or-Col	8
4.2	Parallel	8
4.3	Original Cost Function	11
4.4	Proposed Approach	11
5	Application to KLEIN Family	15
6	Experimental Results	16
6.1	Depth	16
6.2	Quantum Circuits	16
7	Discussion	20
8	Conclusion	21
	References	22
	Appendices	
A	Quantum Circuits	24
A.1	MixColumns	24
A.2	Writing Tips	24
B	Parameters	27
B.1	No. 0001 0050	27
B.2	No. 0051 0100	27

List of Figures

A.1	Quantum Circuit of AES MixColumns for depth 10	25
A.2	Our appaorach for depth 9	26

List of Tables

6.1	Comparison of the depth/gate-count of CNOT circuits for block ciphers . .	16
6.2	Results of the depth/gate-count of CNOT circuits for 32 block ciphers . .	17
6.3	Results of the depth/gate-count of CNOT circuits for 16 block ciphers . .	18
6.4	The implementation of AES MixColumns with quantum depth 9	19

Chapter 1

Introduction

1.1 Motivation

As growing of quantum computer that their computation ability can hazard the current(classical) crypto-system. Such as Shor's algorithm can break the factoring problem, and Grover's Search algorithm can reduce length of security-bit of the symmetric cryptography.

On the other hand, quantum computer can bring new direction for design cryptography and cryptanalysis, we can think physically about computation, this is core spirit in quantum computaiton and quantum information. Specific in quantum cryptography, as the name suggests, it utilizes the cryptographic mechanisms underlying quantum circuits. The benefits that using quantum mechanism is it provided quantum secrecy by uncertainty principle.

To improve efficiency for quantum circuits, it is essential to find more compact implementation for construction. In quantum computing, many application heavily rely on linear reversible circuits; Therefore, deploying efficient methods to reduce the computational cost is key opint to providing more practical algorithms.

Currently, most symmetric cryptosystem are constructed by siginificant number of CNOT gates. Drawing inspiration from the field of Very Large Scale Integration (VLSI), we aim to reduce quantum algorithm costs by focusing on circuit depth as our primary metric, targeting linear scaling.

In this work, we proposed an enhancement randomized greedy algorithm using heuris-
tic methods to achieve the shallowest possible depth with minimum number of CNOT gates. our approach not only guarantees a speedup in the execution of quantum cryptographic algorithms but also reduces the impact of quantum noise and hardware constraints

on the computation results.

1.2 Problem Statement

In this study we address the challenge of finding the optimal (shallowest) depth for constructing the matrix components of symmetric ciphers in gate-based quantum circuit. All matrices are in $\mathbb{F}_2$. Subsequently verify our results can work for real-world block cipher scheme.

1.3 Contributions

Our research focuses on optimizing quantum circuits of block cipher, for instance, those components can bring diffusion and confusion properties for symmetric cryptography, and then we survey latest quantum circuit implementations of lightweight crypto-systems, our improved AES MixColumn (Rijndael) significantly reduces the amounts of CNOT gates for KLEIN family.

1.4 Organization

Rest of this paper: Section 2 we explain all used elements. Section 3 we describe the existing approach for optimal quantum circuit that we considered. Our proposed greedy algorithm is illustrated in Section 4. In Section 5, we will introduce lightweight scheme KLEIN family and prove our optimized circuit can work on it. After all of the methods we mentioned, we would like to show all of the experimental results in Section 6. Finally, Section 7 and 8 are discussion and conclusion for this work.

Chapter 2

Preliminaries

2.1 Notations

In this section we would like show some notations used in this theis are listed as follows:

Notation	Definition
$ 0\rangle, 1\rangle$	
α, β	The probabilies for $ 0\rangle$ and $ 1\rangle$
$ \psi\rangle$	The quantum bit or qubit for short, it is used for describe linear combinations for
$\dagger$	The operation that transpose and then complex conjugating the pauli matrix gate.
$\mathbb{F}_2$	The finite field with two elements 0 and 1
$\oplus$	The XOR computation in logic representation
$E(i + j)$	

2.2 Quantum Circuits

From the Enigma machine to von Neumann classical computer they all consists with wires and logic gates to execute the operations. Similary, quantum computer also need construct by those components but it is combine quantum wires and quantum logic gates to generate quantum circuits which used for transfer the quantum bits and execution the algorithm.

This is the flesh and blood for quantum computer, and using these elemtns to handle

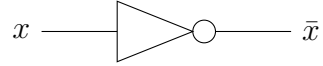
the superposition state through Dirac notation:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad (2.1)$$

In this section, we will introduce some fundamental elements for quantum circuits, including NOT, CNOT and Toffoli gates. Those gates can be understood from matrix operation, and the circuits only generated by these gates we called them NCT-based circuit.

First of all, NOT gate or in Pauli matrix view we called it X gate in general, whose operation can be defined by its truth table, it can make the input bit $0 \rightarrow 1$ and $1 \rightarrow 0$ in classical perspective, but if we switch it to quantum perspective, we can take it as $|0\rangle$ and $|1\rangle$, and it can be acts linearly for $|\psi\rangle$. Through the matrix X the state will change,

$$X \equiv \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$



we think about it as column view

2.3 Optimization Target

In this work, our goal is to reduce the depth of quantum circuits as a metric, which influences the time cost and storage cost for quantum algorithms. After observation, CNOT gate is most used in gate-based quantum computers currently.

And shallow depth of quantum circuits can also make errors decrease, which can reduce the noise that is against quantum decoherence.

2.4 Grover's Search Algorithm

Chapter 3

Existing Approach for CNOT Gate Composition

In this section, we will introduce some existing methods for find the CNOT circuit synthesis that aim to optimal the depth of CNOT circuits.

Building on the work of [4], who proposed an improved depth-oriented algorithm for synthesizing ancilla-free, low-depth CNOT circuits, we developed four distinct greedy strategies to further explore depth-optimized results.

By utilizing different matrix representations and search heuristics, we implemented the Row, Column, Row-or-Column, and Parallel algorithms.

The pseudocode and implementation logic for each approach are detailed in the following sections.

3.1 De Brugière et al. Greedy Method

3.2 Shi & Feng Greedy Method

This strategy aims to drive the matrix toward the 'can_one' condition. Once the 'one' flag is triggered, the algorithm shifts its priority exclusively to row operations to facilitate the final reduction into a permutation matrix.

3.3 Matrix Cost Function

The 'costMat' function serves as our primary heuristic, allowing for the selection of five distinct cost metrics. These metrics—logarithmic, linear sum, square, cubic, and quartic—are controlled by a parameter p , which determines the mathematical weight assigned to the matrix reduction process.

Algorithm 1 Row/Col Greedy

Input: $M \in \mathbb{F}_2^{n \times n}$, $M' \in \mathbb{F}_2^{n \times n}$, $L_r, L_c, L_{s_r}, L_{s_c}, p$ and `over_depth`

Output: $L_{s_r}, L_{s_c}, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length( $M$ )
2: minm  $\leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [\emptyset]$ ,  $visi_{col} \leftarrow [\emptyset]$ 
5: close_permu  $\leftarrow$  False
6: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
7: while  $M \neq$  Permutation do
8:   select_list.clear()
9:    $L_{row}.clear()$ ,  $L_{col}.clear()$ ,  $L_{row}^{cost}.clear()$ ,  $L_{col}^{cost}.clear()$ 
10:   $H_r, H_c, minm \leftarrow$  functionCost( $M, M', p$ )
11:  if !close_permu then
12:     $L_{col} \leftarrow L\_Collection(visi_{row})$ 
13:    select_list, minm  $\leftarrow$  Row_Op_Sel( $L_{col}, L_{col}^{cost}, M, M', p$ )
14:     $L_{row} \leftarrow L\_Collection(visi_{row})$ 
15:    function AVAILABLEROWCOLLECTION( $visi_{row}$ , SIZE)
16:    | return  $L_{row}$ 
17:    function AVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p$ )
18:    | return select_list, minm
19:    function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, L_{s_r}, L_{s_c}, mat,$ 
20:    |  $inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ , SIZE, one)
21:    | return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ , one
22:    if  $d >$  LIMIT then
23:    |  $flag \leftarrow$  True
24:    | return
```

3.3.1 Log

This cost function is derived from the greedy framework proposed by [1], which utilizes the logarithm of the Hamming weight for each row as follows:

$$h_{log}(A) = \sum_i \log_2(\sum_j a_{i,j})$$

3.3.2 Sum

In addition to the logarithmic metric, the greedy framework by [1] considers the cumulative Hamming weight of each row as follows:

$$h_{sum}(A) = \sum_i \sum_j a_{i,j}$$

3.3.3 Square

Following the methodology of [4], this cost function utilizes the square of each row's Hamming weight, defined as follows:

$$h_{sq}(A) = \sum_i (\sum_j a_{i,j})^2$$

Chapter 4

Our Greedy Algorithm

4.1 Row-or-Col

This strategy evaluates both row and column operations and selects the layer that yields the minimum matrix cost. In cases where the costs are identical, the algorithm defaults to the column-wise operation.

If only one set of operations is available, the algorithm selects the existing set.

To address the inherent risk of local minima in greedy heuristics, we implement a stuck threshold. If the cost fails to decrease for three consecutive iterations, the algorithm accepts a sub-optimal operation that increases the cost, thereby enabling an escape from the local minimum.

4.2 Parallel

The Parallel Greedy strategy aggregates both row and column operations into a unified 'select_list'.

In cases where one operation type is unavailable, the algorithm proceeds with the existing candidates. Similar to the Row-or-Column approach, a stagnation threshold is utilized to escape local minima; if no progress is made for three iterations, the algorithm accepts cost-increasing moves.

The primary distinction of the Parallel strategy lies in its late-stage optimization: when the matrix approaches a permutation state, the 'one' flag is activated, restricting the search to column-wise operations to finalize the circuit.

Algorithm 2 Row_or_Col Greedy

Input: $mat_{n \times n}, inv_{n \times n}, L_r, L_c, L_{s_r}, L_{s_c}, p$ and $flag$

Output: $L_{s_r}, L_{s_c}, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length(mat)
2: minm,  $best_{row}^{cost}, best_{col}^{cost} \leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [0], visi_{col} \leftarrow [0]$ 
5: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
6: stuck_counter  $\leftarrow$  0, max_stuck_iterations  $\leftarrow$  3
7: while  $mat \neq$  Permutation do
8:   select_list.clear()
9:    $L_{row}.clear(), L_{col}.clear(), L_{row}^{cost}.clear(), L_{col}^{cost}.clear(), B_{row}.clear(), B_{col}.clear()$ 
10:  if  $p \neq 1$  then
11:     $H_r \leftarrow \text{costMat}(mat, p) + \text{costMat}(inv^T, p)$ 
12:     $H_c \leftarrow \text{costMat}(mat^T, p) + \text{costMat}(inv, p)$ 
13:    minm  $\leftarrow$  max( $H_r, H_c$ )
14:  else
15:    minm  $\leftarrow$  costMat( $mat, p$ ) + costMat( $inv, p$ )
16:  function AVAILABLEROWCOLLECTION( $visi_{row}, SIZE$ )
17:    return  $L_{row}$ 
18:  function MODIFIEDAVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p, minm$ )
19:    return  $B_{row}, best_{row}^{cost}$ 
20:  function AVAILABLECOLCOLLECTION( $visi_{col}, SIZE$ )
21:    return  $L_{col}$ 
22:  function MODIFIEDAVAILABLECOLOPERATORSELECTION( $L_{col}, L_{col}^{cost}, mat, inv, p, minm$ )
23:    return  $B_{col}, best_{col}^{cost}$ 
24:  is_stuck = length( $B_{row}$ ) == 0  $\wedge$  length( $B_{col}$ ) == 0
25:  if is_stuck then
26:    stuck_counter += 1
27:  else
28:    stuck_counter  $\leftarrow$  0
29:  if (stuck_counter  $\geq$  max_stuck_iterations) OR (is_stuck  $\wedge \sum visi_{row} == 0 \wedge \sum visi_{col} == 0$ ) then
30:    escapeCandidates.clear()
31:    function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION( $L_{row}, L_{col}, mat, inv, p,$ 
    escapeCandidates)
32:    return select_list, minm
33:  if  $best_{row}^{cost} < best_{col}^{cost}$  then
34:    select_list  $\leftarrow B_{row}$ 
35:    minm  $\leftarrow best_{row}^{cost}$ 
36:  else if  $best_{row}^{cost} > best_{col}^{cost}$  then
37:    select_list  $\leftarrow B_{col}$ 
38:    minm  $\leftarrow best_{col}^{cost}$ 
39:  else
40:    select_list  $\leftarrow B_{col}$  if len( $B_{col}$ ) > 0 else  $B_{row}$ 
41:    minm  $\leftarrow best_{col}^{cost}$  if len(select_list) > 0 else minm
42:  function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv,$ 
     $row_{op}, col_{op}, visi_{row}, visi_{col}, d, SIZE$ )
43:    return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ 
44:  if  $d \nless$  LIMIT then
45:    flag = True
46:  return
47: return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, row_{op}, col_{op}, d, flag$ 
```

Algorithm 3 Parallel Greedy

Input: $mat_{n \times n}, inv_{n \times n}, L_r, L_c, Ls_r, Ls_c, p$ and $flag$

Output: $Ls_r, Ls_c, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length(mat)
2: minm,  $best_{row}^{cost}, best_{col}^{cost} \leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [0], visi_{col} \leftarrow [0]$ 
5: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
6: one  $\leftarrow$  False
7: stuck_counter  $\leftarrow$  0, max_stuck_iterations  $\leftarrow$  3
8: while  $mat \neq$  Permutation do
9:   select_list.clear()
10:   $L_{row}.clear(), L_{col}.clear(), L_{row}^{cost}.clear(), L_{col}^{cost}.clear(), B_{row}.clear(), B_{col}.clear()$ 
11:  if  $p \neq 1$  then
12:     $H_r = \text{costMat}(mat, p) + \text{costMat}(inv^T, p)$ 
13:     $H_c = \text{costMat}(mat^T, p) + \text{costMat}(inv, p)$ 
14:    minm = max( $H_r, H_c$ )
15:  else
16:    minm = costMat( $mat, p$ ) + costMat( $inv, p$ )
17:  if !one then
18:    function AVAILABLEROWCOLLECTION( $visi_{row}, SIZE$ )
19:    | return  $L_{row}$ 
20:    function MODIFIEDAVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p, minm$ )
21:    | return  $B_{row}, best_{row}^{cost}$ 
22:    function AVAILABLECOLLECTION( $visi_{col}, SIZE$ )
23:    | return  $L_{col}$ 
24:    function MODIFIEDAVAILABLECOLOPERATORSELECTION( $L_{col}, L_{col}^{cost}, mat, inv, p, minm$ )
25:    | return  $B_{col}, best_{col}^{cost}$ 
26:    is_stuck = length( $B_{row}$ ) == 0  $\wedge$  length( $B_{col}$ ) == 0
27:    if is_stuck then
28:      stuck_counter += 1
29:    else
30:      stuck_counter  $\leftarrow$  0
31:    if (stuck_counter  $\geq$  max_stuck_iterations) OR (is_stuck  $\wedge \sum visi_{row} == 0 \wedge \sum visi_{col} == 0$ ) then
32:      escapeCandidates.clear()
33:      function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION( $L_{row}, L_{col}, mat, inv, p,$ 
34:      | return select_list, minm
35:    else
36:      select_list  $\leftarrow B_{row} + B_{col}$ 
37:      if  $best_{row}^{cost} < best_{col}^{cost}$  then
38:        minm  $\leftarrow best_{row}^{cost}$ 
39:      else if  $best_{row}^{cost} > best_{col}^{cost}$  then
40:        minm  $\leftarrow best_{col}^{cost}$ 
41:      else
42:        minm  $\leftarrow best_{col}^{cost}$  if len(select_list) > 0 else minm
43:      function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, Ls_r, Ls_c, mat, inv,$ 
44:      | return  $L_r, L_c, Ls_r, Ls_c, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d, one$ 
45:      if  $d \nless$  LIMIT then
46:        flag  $\leftarrow$  True
47:      return
48: return  $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, d, flag$ 
```

4.3 Original Cost Function

This cost function is derived from the greedy framework proposed by [3], which utilizes the Hamming distance to close to identity matrix for each row as follows:

$$h_{origin}(A) = \sum_{i=1}^n \sum_{j=1}^n (M_{(i,j)} \oplus I_{(i,j)}) + (M_{(i,j)}^{-1} \oplus I_{(i,j)})$$

4.4 Proposed Approach

- M : Given an $n \times n$ input matrix A , our greedy algorithm seeks to identify a sequence of operations that transforms A into a permutation matrix.
- d : The circuit depth achieved when matrix A is transformed into a permutation matrix.
- M' : The matrix cost function is derived from the inverse of matrix A .
- **SIZE**: Since we are specifically considering square block matrices, the matrix size is determined by retrieving the row or column length using the built-in 'length()' function.
- L_r, L_c : Row and column operation sequences are stored in separate layer lists for each iteration.
- L_{s_r}, L_{s_c} : We maintain row-layer and column-layer lists to store the specific operations assigned to each discrete circuit layer.
- L_{row}, L_{col} : A list of remaining operations is updated after each step to exclude those already applied to the matrix.
- $L_{row}^{cost}, L_{col}^{cost}$: For every operation in L_{row} and L_{col} , the algorithm calculates and stores the resulting matrix cost function in a lookup list.
- row_{op}, col_{op} : The collection of applied row and column operations is stored to construct the final sequence list and to verify the correctness of the transformation.

- p : A control parameter is used to select the matrix cost function type. Supported functions include linear sum (1), square (2), cubic (3), quartic (4), and logarithmic (- 1), which can be selected via their respective numerical identifiers.
- `over_depth`: An acceptance flag is utilized to evaluate the current circuit depth; it returns 'True' if the depth d exceeds the current minimum depth, and 'False' otherwise.
- `close_permu`: A termination flag monitors whether the matrix is nearing a permutation state. If the current matrix can be transformed into a permutation matrix within a single layer, the flag is set to True; otherwise, it remains False.
- `minm`: The algorithm identifies the current minimum cost across all candidates and selects the operation corresponding to this optimal value.
- `origin, inverse`: At the start of the process, a copy of the input matrix is retained for final verification. Simultaneously, the matrix inverse is calculated to facilitate the evaluation of the cost function.
- `visirow, visicol`: A visited list tracks the selected row and column indices; once a position is chosen, it is excluded from future iterations to prevent redundant operations.
- `select_list`: The selection list stores the set of candidate operations available for the current optimization step.
- H_r, H_c : The matrix cost function is evaluated from both a row perspective and a column perspective to determine the most efficient operation.
- B_{row}, B_{col} : The algorithm maintains a collection of row and column operations for both the Row/Columnwise and Parallel greedy approaches; it identifies the available operators and selects the one with the lowest computational cost.
- $best_{row}^{cost}, best_{col}^{cost}$: The algorithm maintains a collection of cost values for each potential operation in B_{row} and B_{col} to facilitate the selection of the optimal operator.

- `stuck_counter`: A counter is used to record the number of consecutive stalls (stuck iterations) to detect when the algorithm has reached a local optimum.
- `max_stuck_iteration`: A stagnation threshold is defined to limit the number of consecutive stalls; by default, this value is set to 3.
- `escapeCandidates`: The algorithm also maintains a collection of non-optimal operators, representing candidates whose cost values exceeded the current minimum.
- `best_escape`: To resolve local minima, the algorithm identifies the optimal candidate from the `escapeCandidates` list—after sorting it in ascending order—and adds it to the `select_list` to resume the search.
- `occur_time`: The acceptance counter records the total number of iterations in which an operation was successfully selected and applied.
- `minm_size`: A threshold condition is implemented to ensure that the circuit depth (total number of layers) does not exceed the predefined limit.
- `reduce`: A copy of the initial matrix is retained to verify that the sequence of row and column operations (row_{op} and col_{op}) successfully transforms the origin matrix into the target representation.
- `size_minm`: The algorithm records the final circuit depth to document the efficiency of the optimized sequence.
- `per`: The algorithm records the final matrix state to generate a permutation list, which defines the final mapping of the quantum registers.
- `seq`: The algorithm maintains a comprehensive log of all applied row and column operations (row_{op} and col_{op}), which constitutes the final quantum circuit sequence.
- `layers`: The algorithm maintains a record of the sets LS_c and LS_r , which contain the column and row operations assigned to each parallel layer.
- `correct`: A validation flag is employed to confirm that the combined operation sequence and layer assignments successfully transform the original matrix into a per-

mutation matrix.

- greedy: An algorithmic selection parameter allows the user to specify which greedy strategy (e.g., Row, Column, Row_or_Col or Parallel) should be employed during the synthesis process.

The primary algorithm integrating these cost metrics and greedy heuristics is detailed below:

Algorithm 4 Parameterized Greedy Algorithm

Input: *mat*, *matName*, *greedy*, *p*, *occur_time*

Output: *d*, *Layers*

```
1: minm, minm_size  $\leftarrow$  system.float_info.max
2: SIZE  $\leftarrow$  len(mat)
3: origin.copy(mat)
4: inv  $\leftarrow$  inverse(mat)
5: Lr, Lc, Lsr, Lsc, rowop, colop  $\leftarrow$  list()
6: flag  $\leftarrow$  False
7: function PARAMETERIZEDGREEDY(greedy, p)
8:   return Lr, Lc, Lsr, Lsc, mat, rowop, colop, depth, flag
9: if flag then
10:   return
11: function LASTCHECK(Lr, Lc, Lsr, Lsc)
12:   return Lsr, Lsc
13: reduce.copy(origin)
14: size  $\leftarrow$  0
15: function GENERATEREDUCEMATRIX(reduce, rowop, colop, size)
16:   return reduce, size
17: ok  $\leftarrow$  True
18: if reduce  $\neq$  mat then
19:   ok = False
20: if (d > minm) OR (d == minm  $\wedge$  size  $\geq$  minm_size) OR !ok then
21:   return
22: minm  $\leftarrow$  d
23: size_minm  $\leftarrow$  size
24: per  $\leftarrow$  list()
25: function GENERATEPERMUTATIONLIST(SIZE, mat, per)
26:   return per
27: seq  $\leftarrow$  list()
28: function GENERATESEQUENCELIST(seq, rowop, colop)
29:   return seq
30: layers  $\leftarrow$  list()
31: function GENERATELAYERSLIST(layers, Lsr, Lsc)
32:   return layers
33: correct  $\leftarrow$  Verify(origin, layers, seq, mat)
34: if !correct then
35:   return
36: return d, layers
```

Chapter 5

Application to KLEIN Family

Chapter 6

Experimental Results

6.1 Depth

6.2 Quantum Circuits

Cipher	Size	Q#	[1]	[4]	Our Algorithm
AES	32	30/92	12/128	10/131	9/109
FOX MU4	32	55/136	21/200	21/265	19/191
TWOFISH	32	37/111	18/187	15/175	15/162
WHIRLWIND M0	32	65/183	28/ 286	28/331	25/324
WHIRLWIND M1	32	69/190	25/279	22/290	22/253

Table 6.1: Comparison of the depth/gate-count of CNOT circuits for block ciphers

Cipher	Cost Function	Row	Column	Row-or-Col	Parallel
AES	Log	12/128	12/131	12/131	13/126
	Sum	10/120	10/120	10/121	11/124
	Origin	9/109	9/111	9/109	9/110
ANUBIS	Log	12/126	13/127	12/126	13/140
	Sum	14/133	13/128	13/139	12/136
	Origin	13/138	13/140	14/129	13/129
CLEFIA M0	Log	11/126	11/ 123	11/127	12/149
	Sum	14/138	14/140	13/132	15/145
	Origin	13/136	14/134	12/134	14/139
CLEFIA M1	Log	13/129	13/129	12/129	12/128
	Sum	13/130	12/136	13/128	13/137
	Origin	13/129	13/127	11/123	12/131
FOX MU4	Log	20/266	20/267	21/257	21/221
	Sum	19/196	20/197	19/191	21/212
	Origin	30/313	36/324	30/283	33/327
QARMA128	Log	3/48	3/48	3/48	3/48
	Sum	3/48	3/48	3/48	3/48
	Origin	5/64	4/64	5/64	4/64
TWOFISH	Log	19/231	19/212	19/211	18/232
	Sum	15/162	15/163	16/183	17/178
	Origin	22/246	24/241	22/252	23/263
WHIRLWIND M0	Log	26/329	26/325	26/328	25/324
	Sum	28/294	26/ 293	29/313	32/330
	Origin	39/387	52/425	43/396	43/387
WHIRLWIND M1	Log	25/321	23/291	23/286	22/296
	Sum	23/268	22/265	22/253	23/269
	Origin	48/420	46/420	47/426	51/477
8×8 matrices in $GF(4)$					
ACISP_SarSye17 [2]	Log	29/415	30/404	30/403	29/378
	Sum	28/293	33/321	29/314	29/330
	Origin	40/388	38/363	42/396	36/380
FSE_SKOP15 [5]	Log	29/401	30/398	29/398	29/412
	Sum	32/ 314	31/328	31/317	32/346
	Origin	42/409	38/396	39/393	41/410
FSE_SKOP15 (Involu- tory) [5]	Log	26/337	26/337	26/353	28/373
	Sum	28/300	27/305	31/337	30/ 297
	Origin	40/386	39/372	37/372	37/377

Table 6.2: Results of the depth/gate-count of CNOT circuits for 32 block ciphers

Cipher	Cost Function	Row	Column	Row-or-Col	Parallel
JOLTIK	Log	9/55	9/55	9/55	7/52
	Sum	8/52	8/52	8/52	8/52
	Origin	9/48	8/52	9/48	7/48
MIDORI	Log	12/126	13/127	12/126	13/140
	Sum	14/133	13/128	13/139	12/136
	Origin	13/138	13/140	14/129	13/129
SmallScale AES	Log	11/126	11/123	11/127	12/149
	Sum	14/138	14/140	13/132	15/145
	Origin	13/136	14/134	12/134	14/139
PRIDE L0	Log	13/129	13/129	12/129	12/128
	Sum	13/130	12/136	13/128	13/137
	Origin	13/129	13/127	11/123	12/131
PRIDE L1	Log	20/266	20/267	21/257	21/221
	Sum	19/196	20/197	19/191	21/212
	Origin	30/313	36/324	30/283	33/327
PRIDE L2	Log	3/48	3/48	3/48	3/48
	Sum	3/48	3/48	3/48	3/48
	Origin	5/64	4/64	5/64	4/64
PRIDE L3	Log	19/231	19/212	19/211	18/232
	Sum	15/162	15/163	16/183	17/178
	Origin	22/246	24/241	22/252	23/263
PRINCE M0	Log	26/329	26/325	26/328	25/324
	Sum	28/294	26/293	29/313	32/330
	Origin	39/387	52/425	43/396	43/387
PRINCE M1	Log	25/321	23/291	23/286	22/296
	Sum	23/268	22/265	22/253	23/269
	Origin	48/420	46/420	47/426	51/477
QARMA64	Log	25/321	23/291	23/286	22/296
	Sum	23/268	22/265	22/253	23/269
	Origin	48/420	46/420	47/426	51/477
SKINNY	Log	25/321	23/291	23/286	22/296
	Sum	23/268	22/265	22/253	23/269
	Origin	48/420	46/420	47/426	51/477

Table 6.3: Results of the depth/gate-count of CNOT circuits for 16 block ciphers

Operation	Operation	Operation	Operation	Operation
Depth 1	(15, 31)	(28, 3)	(17, 18)	(9, 25)
(22, 6)	(30, 22)	(21, 29)	Depth 7	(26, 18)
(31, 23)	(24, 16)	(9, 31)	(30, 23)	(8, 0)
(15, 7)	(28, 20)	(14, 30)	(29, 22)	(14, 6)
(30, 14)	(10, 2)	(6, 5)	(10, 3)	(12, 4)
(12, 28)	(25, 9)	(7, 15)	(17, 2)	Depth 9
(29, 13)	(18, 26)	(1, 23)	(15, 12)	(5, 29)
(4, 20)	(0, 8)	(13, 11)	(0, 25)	(21, 13)
(5, 21)	(6, 14)	(20, 22)	(4, 5)	(14, 30)
(2, 18)	(4, 12)	(16, 0)	(8, 9)	(6, 22)
(1, 25)	Depth 3	(27, 2)	(11, 19)	(24, 8)
(8, 24)	(21, 28)	(26, 10)	(31, 20)	(19, 27)
(17, 9)	(19, 18)	Depth 5	Depth 8	(3, 11)
(27, 19)	(20, 27)	(25, 15)	(13, 29)	(7, 15)
(11, 3)	(14, 13)	(28, 23)	(7, 23)	(23, 31)
(10, 26)	(7, 6)	(19, 31)	(1, 17)	(18, 2)
(0, 16)	(8, 15)	Depth 6	(27, 11)	(28, 12)
Depth 2	(26, 1)	(22, 31)	(21, 5)	(25, 1)
(23, 7)	(16, 31)	(4, 20)	(31, 15)	(20, 4)
(13, 21)	(24, 23)	(15, 3)	(19, 3)	(26, 10)
(29, 5)	(9, 17)	(24, 9)	(22, 30)	(9, 17)
(3, 19)	(12, 11)	(12, 13)	(16, 24)	(16, 0)
(17, 1)	Depth 4	(23, 27)	(20, 28)	
(11, 27)	(18, 25)	(8, 1)	(2, 10)	

Table 6.4: The implementation of AES MixColumns with quantum depth 9

Chapter 7

Discussion

Chapter 8

Conclusion

References

- [1] Timothée Goubault de Brugière et al. “Reducing the Depth of Linear Reversible Quantum Circuits”. In: *IEEE Transactions on Quantum Engineering* 2 (2021), pp. 1–22. DOI: [10.1109/TQE.2021.3091648](https://doi.org/10.1109/TQE.2021.3091648).
- [2] Sumanta Sarkar and Habeeb Syed. *Analysis of Toeplitz MDS Matrices*. Cryptology ePrint Archive, Paper 2017/368. 2017. URL: <https://eprint.iacr.org/2017/368>.
- [3] Ben Schaeffer and Marek Perkowski. “A cost minimization approach to synthesis of linear reversible circuits”. In: *arXiv preprint arXiv:1407.0070* (2014).
- [4] Haotian Shi and Xiutao Feng. *Quantum Circuits of AES with a Low-depth Linear Layer and a New Structure*. Cryptology ePrint Archive, Paper 2024/381. 2024. URL: <https://eprint.iacr.org/2024/381>.
- [5] Siang Meng Sim et al. *Lightweight MDS Involution Matrices*. Cryptology ePrint Archive, Paper 2015/258. 2015. URL: <https://eprint.iacr.org/2015/258>.

Appendices

Appendix A

Quantum Circuits

In this section, we would like to show the whole quantum circuits using Drawin-
gEngine.

A.1 MixColumns

A.2 Writing Tips

- Include only materials that aid understanding but are too lengthy for the main text.
- Ensure appendices follow the same formatting standards as the main document.
- De-identify data, especially for human subjects or sensitive information, to comply with ethical standards.

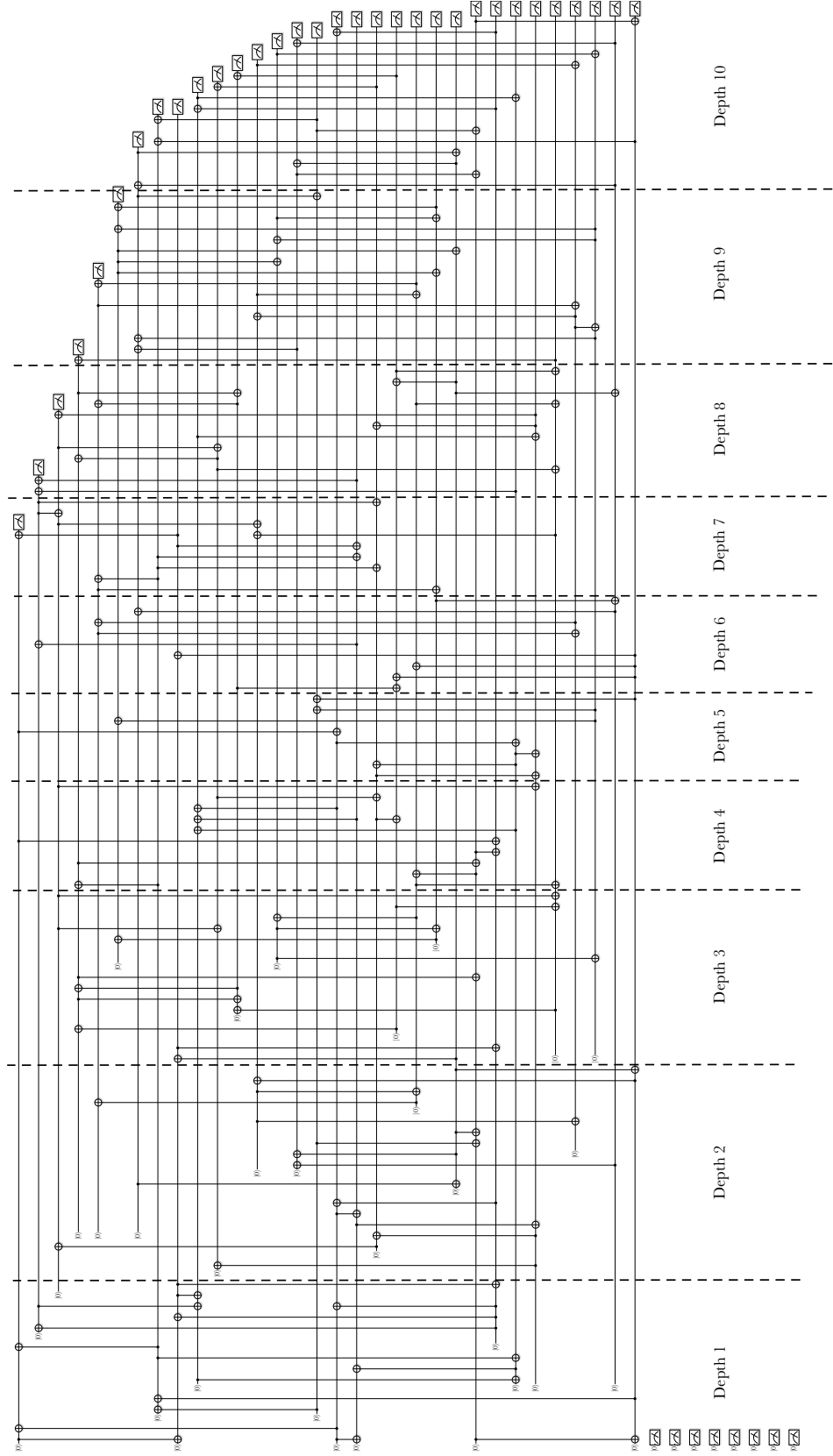


Fig. A.1: Quantum Circuit of AES MixColumns for depth 10

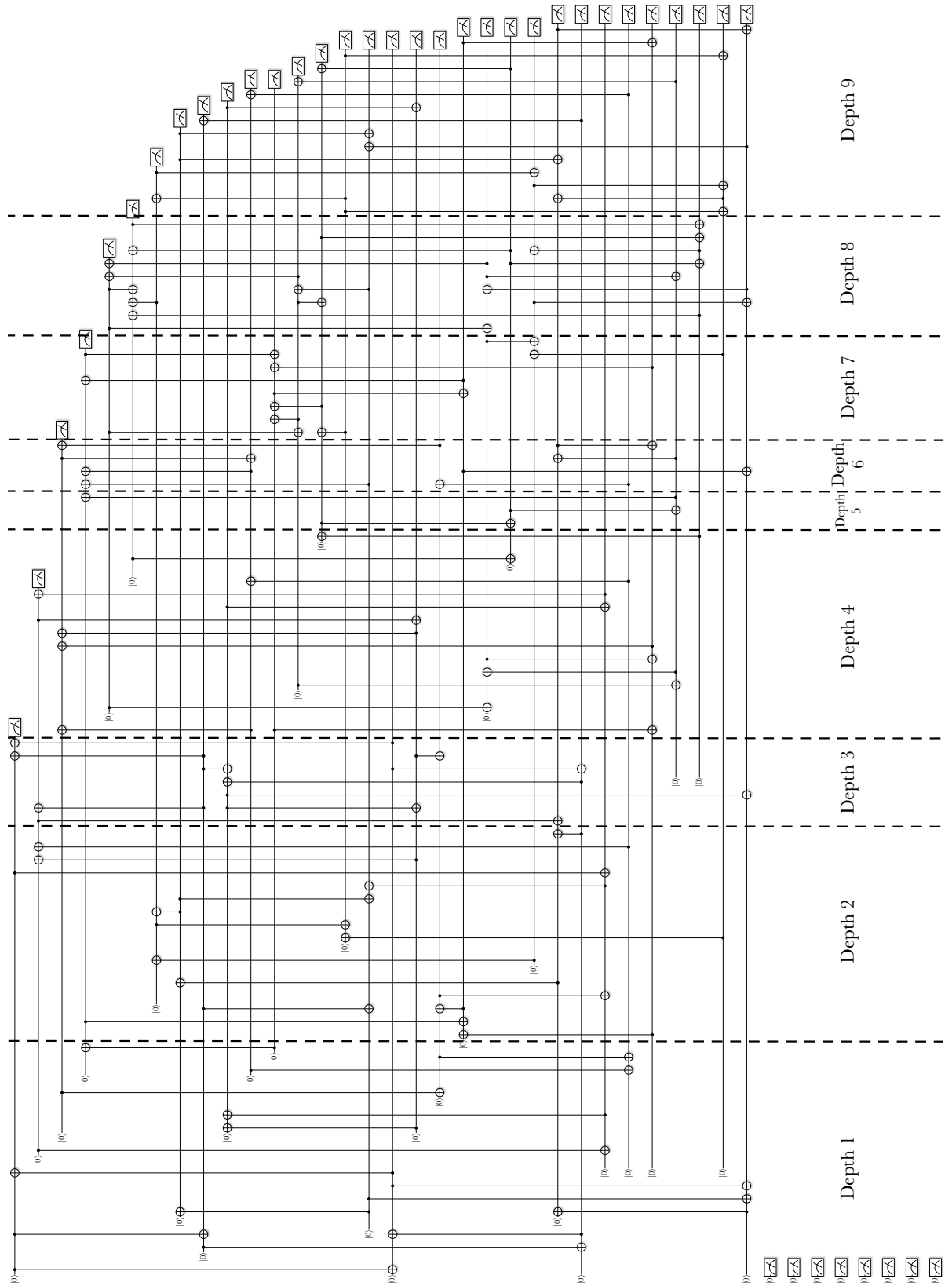


Fig. A.2: Our appaarach for depth 9

Appendix B

Parameters

B.1 No. 0001 0050

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

B.2 No. 0051 0100

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.